

FUZZY HYBRID: TSUKAMOTO, MAMDANI, SUGENO

1. Fuzzy Tsukamoto

Pada metode Tsukamoto, setiap aturan fuzzy menghasilkan output crisp (tegas) berdasarkan fungsi keanggotaan monoton. Output akhir diperoleh dengan rata-rata tertimbang dari semua output aturan.

Ciri khas: setiap aturan menghasilkan *output crisp* (tegas), karena fungsi keanggotaan konsekuen harus *monoton*. Hasil akhir diperoleh dengan **rata-rata terbobot**.

Contoh sehari-hari

- **Sistem Prediksi Tingkat Kesegaran Buah**
 - Input: warna kulit buah, kelembutan, dan kadar air.
 - Aturan:
 - IF warna hijau AND tekstur keras THEN kesegaran tinggi.
 - IF warna coklat OR tekstur lembek THEN kesegaran rendah.
 - Dengan Tsukamoto, tiap aturan menghasilkan angka tegas (misalnya kesegaran = 80%).

Contoh skripsi/penelitian

- **Sistem Pendukung Keputusan Kelayakan Kredit**
 - Input: gaji, umur, lama bekerja.
 - Output: tingkat kelayakan (angka tegas 0–100).
 - Metode Tsukamoto dipilih karena outputnya harus jelas (layak/tidak layak).

Contoh Kode Python Tsukamoto:

```
def tsukamoto(x, a, b):  
    # Fungsi keanggotaan linear naik  
    if x <= a:  
        return 0  
    elif x >= b:  
        return 1  
    else:  
        return (x - a) / (b - a)  
  
# Contoh: x = 60, a = 50, b = 100  
x = 60  
a = 50  
b = 100
```

```
mu = tsukamoto(x, a, b)
print(f"Derajat keanggotaan Tsukamoto: {mu}")
```

Pseudocode

```
FUNCTION tsukamoto(x, a, b):
  IF x <= a THEN
    RETURN 0
  ELSE IF x >= b THEN
    RETURN 1
  ELSE
    RETURN (x - a) / (b - a)
  END IF
END FUNCTION

SET x = 60
SET a = 50
SET b = 100

mu = tsukamoto(x, a, b)

PRINT "Derajat keanggotaan Tsukamoto: ", mu
```

Penjelasan:

Bayangkan kita sedang menilai **tingkat kepantasan nilai mahasiswa** untuk dikategorikan sebagai *cukup pintar*.

Misalnya, batas bawah "cukup pintar" adalah nilai **50** dan batas atasnya adalah **100**.

- Jika seorang mahasiswa mendapat nilai ≤ 50 , maka dia **belum dianggap cukup pintar** → derajat keanggotaan **0**.
- Jika nilainya ≥ 100 , maka dia **sudah pasti cukup pintar** → derajat keanggotaan **1**.
- Jika nilainya berada **di antara 50 dan 100**, maka kepintaran mahasiswa tersebut **bersifat linier** (semakin besar nilainya, semakin tinggi kepantasan "cukup pintar").

Contohnya:

Seorang mahasiswa mendapat nilai **60**.

Dengan aturan Tsukamoto:

$$\mu = \frac{60 - 50}{100 - 50} = \frac{10}{50} = 0.2$$

Artinya, mahasiswa ini memiliki derajat keanggotaan **0.2** pada kategori *cukup pintar*.

Bukan berarti dia tidak pintar sama sekali, tapi **hanya 20% tingkat kepantasan** untuk dianggap cukup pintar, sisanya masih perlu ditingkatkan.

2. Fuzzy Mamdani

Metode Mamdani menggunakan aturan IF-THEN dengan output berupa himpunan fuzzy. Proses defuzzifikasi (misal: centroid) digunakan untuk mendapatkan output crisp.

Ciri khas: menggunakan fungsi keanggotaan fuzzy pada output, sehingga hasil inferensi berupa **fuzzy set**, lalu dilakukan **defuzzifikasi** (misalnya *centroid method*).

Contoh sehari-hari

- **Sistem Pengatur Pendingin Ruangan (AC)**
 - Input: suhu ruangan & jumlah orang.
 - Aturan:
 - IF suhu panas AND banyak orang THEN kecepatan kipas tinggi.
 - IF suhu dingin AND sedikit orang THEN kecepatan kipas rendah.
 - Mamdani cocok karena output berupa fuzzy (lambat, sedang, cepat) → kemudian dikonversi jadi nilai motor kipas.

Contoh skripsi/penelitian

- **Pengendalian Kecepatan Motor DC**
 - Input: error (perbedaan kecepatan) & perubahan error.
 - Output: tegangan motor.
 - Mamdani sering dipilih di penelitian teknik kontrol karena lebih mudah dipahami oleh manusia (aturan berbentuk linguistik).

Contoh kode Python

```
import numpy as np
import skfuzzy as fuzz
import skfuzzy.control as ctrl

# Variabel input dan output
suhu = ctrl.Antecedent(np.arange(0, 101, 1), 'suhu')
kipas = ctrl.Consequent(np.arange(0, 101, 1), 'kipas')

# Himpunan fuzzy
suhu['dingin'] = fuzz.trimf(suhu.universe, [0, 0, 50])
suhu['panas'] = fuzz.trimf(suhu.universe, [50, 100, 100])
kipas['lambat'] = fuzz.trimf(kipas.universe, [0, 0, 50])
kipas['cepat'] = fuzz.trimf(kipas.universe, [50, 100, 100])

# Aturan fuzzy
rule1 = ctrl.Rule(suhu['dingin'], kipas['lambat'])
rule2 = ctrl.Rule(suhu['panas'], kipas['cepat'])
```

```
# Sistem kontrol
kipas_ctrl = ctrl.ControlSystem([rule1, rule2])
kipas_simulasi = ctrl.ControlSystemSimulation(kipas_ctrl)

# Input dan output
kipas_simulasi.input['suhu'] = 70
kipas_simulasi.compute()
print(f"Kecepatan kipas (Mamdani): {kipas_simulasi.output['kipas']}")
```

Pseudocode

```
Mulai program
Buat variabel fuzzy untuk input: suhu (0 – 100)
Buat variabel fuzzy untuk output: kipas (0 – 100)

Definisikan himpunan fuzzy:
- suhu dingin = segitiga [0, 0, 50]
- suhu panas = segitiga [50, 100, 100]
- kipas lambat = segitiga [0, 0, 50]
- kipas cepat = segitiga [50, 100, 100]

Tentukan aturan fuzzy:
- Jika suhu dingin maka kipas lambat
- Jika suhu panas maka kipas cepat

Buat sistem kontrol berdasarkan aturan fuzzy
Buat simulasi kontrol

Masukkan nilai input suhu = 70
Hitung kecepatan kipas berdasarkan logika fuzzy
Tampilkan hasil kecepatan kipas

Selesai
```

Penjelasan

Bayangkan kita sedang mendesain **kipas otomatis** di sebuah ruangan.

Tujuan kita adalah membuat kipas bisa **menyesuaikan kecepatan** sesuai dengan suhu ruangan, tapi tidak menggunakan logika "keras" (if-else), melainkan memakai **logika fuzzy** yang lebih fleksibel.

1. **Definisi variabel**

Kita definisikan **suhu ruangan** (0–100 derajat) sebagai **input**, dan **kecepatan kipas** (0–100 persen) sebagai **output**.

2. **Himpunan fuzzy**

Kita bagi suhu menjadi dua kategori:

- **Dingin** → kalau suhu mendekati 0 derajat, makin jelas dingin.
- **Panas** → kalau suhu mendekati 100 derajat, makin jelas panas.

Begitu juga kecepatan kipas:

- **Lambat** → kipas berputar pelan.
- **Cepat** → kipas berputar kencang.

Jadi, tidak ada batas kaku (misalnya "suhu > 50 pasti panas"), tapi lebih ke **tingkat keanggotaan**: semakin tinggi suhunya, semakin "panas" nilainya.

3. Aturan fuzzy

Kita buat aturan sederhana:

- Jika **suhu dingin** → kipas **lambat**
- Jika **suhu panas** → kipas **cepat**

Aturan ini seperti pengetahuan sehari-hari manusia tentang kipas.

4. Simulasi

Kita coba masukkan **suhu = 70**.

- Pada suhu 70, derajat keanggotaan "dingin" sudah kecil, tapi "panas" cukup besar.
- Aturan "suhu panas → kipas cepat" lebih dominan.
- Hasil akhirnya, kipas berputar dengan kecepatan tinggi (nilai fuzzy yang dihitung dengan metode Mamdani).

5. Output

Program mencetak hasil kecepatan kipas.

Misalnya, untuk suhu 70, kipas mungkin berputar dengan kecepatan sekitar 70–80 (dari skala 0–100).

Jadi intinya, kode ini adalah **sistem logika fuzzy Mamdani sederhana** untuk mengontrol kipas berdasarkan suhu: semakin panas, semakin cepat kipas berputar.

3. Fuzzy Sugeno

Pada metode Sugeno, output dari setiap aturan adalah fungsi linear atau konstan dari input. Output akhir diperoleh dengan rata-rata tertimbang.

Ciri khas: konsekuen berbentuk persamaan linear atau konstanta → hasil akhir langsung berupa **angka tegas** (mirip *regresi*).

Contoh sehari-hari

- **Sistem Prediksi Harga Rumah**

- Input: luas bangunan & lokasi.

- Aturan:
 - IF lokasi strategis AND luas besar THEN harga = $1.5 \times \text{luas} + 500$ juta.
 - IF lokasi biasa AND luas kecil THEN harga = $1.2 \times \text{luas} + 200$ juta.
- Output langsung berupa harga tegas (misal Rp 850 juta).

Contoh skripsi/penelitian

- **Prediksi Lama Studi Mahasiswa**
 - Input: IPK semester awal & jumlah SKS tiap semester.
 - Output: prediksi lama studi (tahun).
 - Sugeno dipilih karena fleksibel untuk integrasi dengan metode lain (misalnya ANFIS = Sugeno + Neural Network).

Contoh kode Python

```
def sugeno(x, y):
    # Aturan 1: IF x kecil AND y kecil THEN z = 10
    # Aturan 2: IF x besar OR y besar THEN z = 20
    mu1 = min(1 - x, 1 - y)
    mu2 = max(x, y)
    z1 = 10
    z2 = 20
    z = (mu1 * z1 + mu2 * z2) / (mu1 + mu2)
    return z

# Contoh: x = 0.3, y = 0.7
x = 0.3
y = 0.7
output = sugeno(x, y)
print(f"Output Sugeno: {output}")
```

Pseudocode

```
FUNCTION Sugeno(x, y):
    # Definisi aturan fuzzy
    # Rule 1: IF x kecil AND y kecil THEN z = 10
    # Rule 2: IF x besar OR y besar THEN z = 20

    mu1 ← MIN(1 - x, 1 - y)  # Derajat keanggotaan untuk aturan 1
    mu2 ← MAX(x, y)          # Derajat keanggotaan untuk aturan 2

    z1 ← 10                  # Output aturan 1
    z2 ← 20                  # Output aturan 2
```

```
# Hitung nilai akhir dengan weighted average (Sugeno)
```

```
z ← (mu1 * z1 + mu2 * z2) / (mu1 + mu2)
```

```
RETURN z
```

```
# Main Program
```

```
SET x = 0.3
```

```
SET y = 0.7
```

```
output ← Sugeno(x, y)
```

```
PRINT "Output Sugeno: ", output
```

Penjelasan

Bayangkan kamu sedang membuat **sistem fuzzy Sugeno sederhana** untuk menilai suatu kondisi berdasarkan dua input: x dan y.

- Pertama, kamu punya **dua aturan**:
 1. Jika **x kecil** dan **y kecil**, maka hasil z condong ke **10**.
 2. Jika **x besar** atau **y besar**, maka hasil z condong ke **20**.
- Karena sistem ini fuzzy, tidak ada jawaban mutlak "ya" atau "tidak". Maka digunakan **derajat keanggotaan**:
 - Untuk aturan 1, dipakai $\mu_1 = \min(1 - x, 1 - y) \rightarrow$ semakin kecil x dan y, semakin tinggi nilai aturan ini.
 - Untuk aturan 2, dipakai $\mu_2 = \max(x, y) \rightarrow$ semakin besar salah satu x atau y, semakin kuat aturan ini berlaku.
- Selanjutnya, sistem memberi **nilai konsekuen**:
 - Aturan 1 menghasilkan angka **10**.
 - Aturan 2 menghasilkan angka **20**.
- Lalu dihitung dengan **rata-rata berbobot** (weighted average ala Sugeno):

$$z = (\mu_1 * z_1 + \mu_2 * z_2) / (\mu_1 + \mu_2)$$

sehingga hasil akhir bukan sekadar 10 atau 20, tapi **nilai di antara keduanya**, tergantung seberapa kuat masing-masing aturan berlaku.

Misalnya ketika $x = 0.3$ dan $y = 0.7$:

- Aturan 1 masih punya sedikit pengaruh (karena x cukup kecil, tapi y tidak kecil).
- Aturan 2 punya pengaruh lebih besar (karena y cukup besar).
 $\rightarrow$ Hasil akhirnya adalah sebuah angka kompromi antara 10 dan 20.

Program akhirnya mencetak:

Output Sugeno: 16.923076923076923

Perbandingan

Metode	Output	implementasi	contoh
Tsukamoto	Crisp tiap aturan	SPK, penilaian, klasifikasi	Kelayakan kredit, kesegaran buah
Mamdani	Fuzzy → defuzzifikasi	Kontrol, sistem real-time	Pengendalian motor, AC otomatis
Sugeno	Linear/konstanta	Prediksi, integrasi ML	Prediksi harga, prediksi lama studi

Catatan

Crisp tiap aturan (Tsukamoto)

- **Crisp** = nilai tegas (angka pasti, bukan himpunan fuzzy).
- Pada **Fuzzy Tsukamoto**, setiap aturan IF–THEN tidak menghasilkan himpunan fuzzy, tetapi langsung angka tegas.
- Caranya: setiap aturan punya **fungsi keanggotaan monotonik** (misalnya naik/turun). Setelah dihitung, keluar satu nilai pasti (misalnya *kelayakan* = 65 dari 100).
- **Analoginya:**
 - Aturan: *IF penghasilan tinggi AND usia muda THEN kelayakan kredit tinggi.*
 - Hasil aturan = angka pasti, misalnya **72.5** (bukan “tinggi”, tapi langsung angka).

Fuzzy → Defuzzifikasi (Mamdani)

- Pada **Mamdani**, hasil tiap aturan masih dalam bentuk **himpunan fuzzy** (misalnya: kecepatan kipas = “sedang” dengan derajat 0.6, “cepat” dengan derajat 0.3).
- Jadi outputnya masih *abu-abu* (fuzzy set).
- Supaya bisa dipakai di dunia nyata, harus diubah jadi angka tegas → proses ini disebut **defuzzifikasi** (metode populer: *centroid, mean of maximum*).
- **Analoginya:**
 - Hasil fuzzy: kipas “sedang” (0.6) + “cepat” (0.3).
 - Setelah defuzzifikasi: keluar nilai tegas, misalnya **65% kecepatan kipas**.

Linear/konstanta (Sugeno)

- Pada **Sugeno**, konsekuen (bagian THEN) berupa **rumus matematis (linear)** atau **angka konstan**.
 - **Konstanta:** *IF x kecil AND y besar THEN z = 10.*

- **Linear:** *IF x kecil AND y besar THEN $z = 2x + 3y + 5$.*
- Artinya output tiap aturan langsung angka (crisp), tapi cara mendapatkannya lewat **rumus matematis**.
- **Analoginya:**
 - Aturan: *IF jarak dekat AND bahan bakar sedikit THEN konsumsi = $0.8 \times \text{jarak} + 2$.*
 - Hasilnya langsung angka (misalnya **9.6 liter**).

IMPLEMENTASI LENGKAP SISTEM FUZZY

Contoh Sistem Fuzzy untuk Kontrol Suhu

```
import numpy as np
import matplotlib.pyplot as plt

class FuzzySystem:
    def __init__(self):
        self.temperature_range = np.arange(0, 101, 1)
        self.speed_range = np.arange(0, 101, 1)

    def membership_function(self, x, a, b, c, membership_type='triangle'):
        if membership_type == 'triangle':
            if x <= a or x >= c:
                return 0
            elif a < x <= b:
                return (x - a) / (b - a)
            else:
                return (c - x) / (c - b)
        elif membership_type == 'trapezoid':
            if x <= a or x >= d:
                return 0
            elif a < x <= b:
                return (x - a) / (b - a)
            elif b < x <= c:
                return 1
            else:
                return (d - x) / (d - c)

    def tsukamoto_method(self, temperature):
        # Definisi himpunan fuzzy untuk suhu
        cold = self.membership_function(temperature, 0, 0, 30)
        warm = self.membership_function(temperature, 20, 50, 80)
        hot = self.membership_function(temperature, 70, 100, 100)

        # Aturan fuzzy
        # R1: IF suhu dingin THEN kecepatan lambat
        # R2: IF suhu hangat THEN kecepatan sedang
        # R3: IF suhu panas THEN kecepatan cepat

        # Output crisp untuk setiap aturan
        z1 = 20 # lambat
        z2 = 50 # sedang
        z3 = 80 # cepat
```

```

# Rata-rata tertimbang
numerator = cold * z1 + warm * z2 + hot * z3
denominator = cold + warm + hot

if denominator == 0:
    return 50 # default value

return numerator / denominator

def mamdani_method(self, temperature):
    # Implementasi Mamdani dengan defuzzifikasi centroid
    # (Disederhanakan untuk contoh)
    cold = self.membership_function(temperature, 0, 0, 30)
    warm = self.membership_function(temperature, 20, 50, 80)
    hot = self.membership_function(temperature, 70, 100, 100)

    # Defuzzifikasi dengan weighted average
    speeds = [20, 50, 80]
    memberships = [cold, warm, hot]

    numerator = sum(s * m for s, m in zip(speeds, memberships))
    denominator = sum(memberships)

    if denominator == 0:
        return 50

    return numerator / denominator

def sugeno_method(self, temperature):
    # Implementasi Sugeno dengan fungsi linear
    cold = self.membership_function(temperature, 0, 0, 30)
    warm = self.membership_function(temperature, 20, 50, 80)
    hot = self.membership_function(temperature, 70, 100, 100)

    # Fungsi output linear
    # z1 = 10 + 0.5*temperature (untuk dingin)
    # z2 = 20 + 0.6*temperature (untuk hangat)
    # z3 = 30 + 0.7*temperature (untuk panas)

    z1 = 10 + 0.5 * temperature
    z2 = 20 + 0.6 * temperature
    z3 = 30 + 0.7 * temperature

    numerator = cold * z1 + warm * z2 + hot * z3
    denominator = cold + warm + hot

    if denominator == 0:

```

```

        return 50

    return numerator / denominator

# Penggunaan sistem fuzzy
fuzzy_system = FuzzySystem()

# Test dengan berbagai suhu
temperatures = [10, 25, 40, 60, 85, 95]

print("Hasil Perbandingan Metode Fuzzy:")
print("Suhu\tTsukamoto\tMamdani\tSugeno")
print("-" * 50)

for temp in temperatures:
    tsukamoto_result = fuzzy_system.tsukamoto_method(temp)
    mamdani_result = fuzzy_system.mamdani_method(temp)
    sugeno_result = fuzzy_system.sugeno_method(temp)

    print(f"{temp}°C\t{tsukamoto_result:.2f}\t\t{mamdani_result:.2f}\t\t{sugeno_result:.2f}")

```

REFERENSI

- Kusumadewi, S. (2002). Artificial Intelligence (Teknik dan Aplikasinya). Yogyakarta: Graha Ilmu.
- Ross, T. J. (2010). Fuzzy Logic with Engineering Applications. Wiley.
- <https://pythonhosted.org/scikit-fuzzy/>

CATATAN PENTING

1. Tsukamoto : Cocok untuk sistem dengan output yang sudah diketahui secara pasti
2. Mamdani : Lebih intuitif dan mudah dipahami, cocok untuk sistem kontrol
3. Sugeno : Lebih efisien komputasi, cocok untuk sistem yang membutuhkan kecepatan tinggi

Semua metode fuzzy ini dapat dikombinasikan (hybrid) untuk mendapatkan performa optimal sesuai kebutuhan aplikasi.